LLD Phase 5 — PDF 1

Tic-Tac-Toe · Vending Machine · Library Management · Parking Lot | Requirements first, hints in the middle, full solutions at the back

> **Colour key. Blue = a note or hint · Red = a common mistake · Green = a rule to remember · Amber = how to say it in an interview.**

Contents

Before you start — how to use this PDF · time boxes · warm-up: 14 requirements, name the pattern

Problem 1 — Tic-Tac-Toe (small)

Problem 2 — Vending Machine (core)

Problem 3 — Library Management (core)

Problem 4 — Parking Lot (heavy)

Hints — three levels per problem: a nudge, the shape, the key decision

Solutions 1–4 — clarifying questions · Part A model answer · full code · Part C answers · common mistakes · 10-point checklist

Back pages — warm-up answers · mistake log · re-do schedule

Before you start

Keep the **Starter Kit** open next to this PDF: it has the method, the pattern cheat sheet and the instructions for checking your output. The loop for each problem is the one from the kit:

#	Do
1	Read the problem once. Start a timer for the time box.
2	**Part A** on paper, out loud: entities, relationships, a diagram, patterns with reasons, failure and concurrency decisions.
3	Stuck for 15 minutes? Read **one** hint level, then go back to work.
4	**Part B** in your IDE. Run it on the given input and compare with the expected output until they match exactly.
5	**Part C** out loud. Open every "defend it" answer with a concession.
6	Only then read the solution. Score yourself with its checklist, and write your mistakes in the log at the back.

Problem	Size	Time box	Format
1. Tic-Tac-Toe	small	45 min	10 design + 35 code
2. Vending Machine	core	75 min	20 design + 55 code
3. Library Management	core	75 min	20 design + 55 code
4. Parking Lot	heavy	100 min	25 design + 75 code

> **If you go over the time box,** finish anyway, but write down by how much. The time box matters more in later PDFs; for now, getting it right matters more than getting it fast.

Warm-up: name the pattern (10 minutes)

All of these come from one app, a gym chain in Pune. For each, name the pattern, or say "nothing" and what you'd do instead. The answers are on the back pages.

1. Monthly, quarterly and yearly plans each have their own discount formula, and marketing changes the formulas every January.
2. When a class is cancelled: text the booked members, refund them, and update the trainer's schedule. The cancellation stands even if the SMS service is down.
3. A personal-training request moves through REQUESTED → CONFIRMED → COMPLETED, or CANCELLED. What you can do depends on the status, for every one of five operations.
4. Members who haven't visited in 30 days get a "we miss you" message.
5. The government fitness-subsidy SDK offers `submitClaim(Map<String, String>)`. Our code calls `SubsidyService.claim(Member)` everywhere.
6. A protein shake can have extra whey, peanut butter and banana, in any combination, each adding to the price.
7. Refunds up to Rs 500 are approved at the front desk, up to Rs 5,000 by the manager, and above that by the owner.
8. Signing up a member: create the account, assign a locker, take the first payment, print the ID card. The website, the app and the front desk all do this.
9. A workout plan has a name and a goal (required) and up to ten optional settings. Once created, it never changes.
10. Each locker has a number and a size.
11. The trainer's app has "Repeat last week's plan", which copies the plan so it can be edited.
12. On the equipment-booking screen, choosing "Treadmill" shows the speed box and hides the weight box, and ticking "With trainer" shows the trainer list.
13. The equipment types are fixed (treadmill, rack, cycle), but finance keeps asking for new monthly reports over them: maintenance cost, usage hours, depreciation.
14. For members without a personal trainer, the "trainer notes" section should simply show nothing.

Problem 1 — Tic-Tac-Toe

Small · 45 minutes (10 design + 35 code) · machine-coding format

The story. The IT office at Hinjewadi wants a console Tic-Tac-Toe for its break room. It starts on the usual 3 × 3 board, but the team already wants 4 × 4 and 5 × 5 games, and someone has asked about 10 × 10.

Functional requirements

R1	A game is played on an N × N board, where N is at least 3.
R2	There are two players. The first plays X and the second plays O. X always moves first.
R3	Players take turns placing their symbol on an empty cell. Rows and columns are numbered from 1.
R4	A move outside the board, or onto a taken cell, is rejected, and the same player tries again.
R5	A player who fills a whole row, a whole column or either diagonal wins, and the game ends.
R6	If the board fills up with no winner, the game is a draw, and it ends.
R7	Once the game has ended, further moves are rejected.
R8	The board can be printed at any time. Starting a new game replaces the old one.

Non-functional requirements

NF1	Boards may become large (100 × 100). After a move, checking for a win must not scan the whole board.
NF2	A different win rule later (for example "5 in a row wins on a 10 × 10 board") should not mean rewriting the game's turn logic.

Out of scope: a computer opponent, undo, saving games, network play.

Commands and exact output

Command	Output
`START n name1 name2`	`Game started: 3x3, Asha (X) vs Ravi (O)`
`MOVE row col`	`Asha placed X at (1,1)`. If that move ends the game, one more line: `Asha wins!` or `It's a draw!`
`PRINT`	One line per row. Cells are separated by one space: `X`, `O`, or `.` for empty.

Error (checked in this order for MOVE)	Output
`MOVE or PRINT before any START`	`No game in progress`
`START with n < 3`	`Board size must be at least 3`
`MOVE after the game has ended`	`Game is over`
`MOVE outside the board`	`Invalid move: (4,2) is outside the board`
`MOVE onto a taken cell`	`Invalid move: (1,1) is already taken`

Part A — design (on paper, out loud)

1	List the entities. For each one, say whether it's a class, an enum, a record or just a field, and why.
2	Who owns each of these: "is this cell empty?", "whose turn is it?", "did that move win?" One line each.
3	Does this problem need a design pattern? Name it, or say why not.
4	NF1: how do you check for a win after a move without scanning the whole board?

Part B — code

Write a single `Main.java` that reads the commands from standard input. Run it on this input, and compare your output with the expected output, line by line.

input.txt

```
MOVE 1 1
START 2 Asha Ravi
START 3 Asha Ravi
MOVE 1 1
MOVE 1 1
MOVE 4 2
MOVE 1 2
MOVE 2 2
MOVE 1 3
PRINT
MOVE 3 3
MOVE 2 1
START 3 Meera Kabir
MOVE 1 1
MOVE 2 2
MOVE 1 3
MOVE 1 2
MOVE 3 2
MOVE 3 1
MOVE 2 1
MOVE 2 3
MOVE 3 3
PRINT
START 4 Asha Ravi
MOVE 1 2
MOVE 1 1
MOVE 2 2
MOVE 2 1
MOVE 3 2
MOVE 3 1
MOVE 4 2
```

expected output

```
No game in progress
Board size must be at least 3
Game started: 3x3, Asha (X) vs Ravi (O)
Asha placed X at (1,1)
Invalid move: (1,1) is already taken
Invalid move: (4,2) is outside the board
Ravi placed O at (1,2)
Asha placed X at (2,2)
Ravi placed O at (1,3)
X O O
. X .
. . .
Asha placed X at (3,3)
Asha wins!
Game is over
Game started: 3x3, Meera (X) vs Kabir (O)
Meera placed X at (1,1)
Kabir placed O at (2,2)
Meera placed X at (1,3)
Kabir placed O at (1,2)
Meera placed X at (3,2)
Kabir placed O at (3,1)
Meera placed X at (2,1)
Kabir placed O at (2,3)
Meera placed X at (3,3)
It's a draw!
X O X
X O O
O X X
Game started: 4x4, Asha (X) vs Ravi (O)
Asha placed X at (1,2)
Ravi placed O at (1,1)
Asha placed X at (2,2)
Ravi placed O at (2,1)
Asha placed X at (3,2)
Ravi placed O at (3,1)
Asha placed X at (4,2)
Asha wins!
```

Part C — pressure questions (out loud)

1	**Change.** "Add an UNDO command that takes back the last move." What do you store, and what must undo restore besides the cell?
2	**Change.** "On a 10 × 10 board, 5 in a row wins." Which part of your design changes, and which doesn't?
3	**Performance.** "Can you check for a win in constant time per move?" How?
4	**Defend it.** "Why separate `Board` and `Game`? One class would be simpler."

Problem 2 — Vending Machine

Core · 75 minutes (20 design + 55 code) · machine-coding format

The story. A snack vending machine stands on platform 1 at Pune Junction. Travellers buy in a hurry, often with whatever coins they have. An operator restocks it twice a day and loads coins for change.

Functional requirements

R1	The machine has slots (A1, A2, B1, ...). Each slot holds one product with a name, a price in whole rupees, and a quantity.
R2	It accepts only Rs 1, 2, 5, 10 and 20. Anything else is rejected.
R3	To buy, the customer first selects a slot, then inserts money one coin or note at a time. As soon as the money covers the price, the product drops and the change comes out.
R4	Change comes from the machine's coin stock, **largest denominations first**. The inserted money joins the stock when the purchase completes, and may itself be used as change.
R5	If exact change can't be made that way, the purchase is cancelled and all the inserted money is refunded. The coin stock and the product quantity stay exactly as they were.
R6	The customer can cancel at any time before the product drops. All inserted money is refunded.
R7	The operator switches **service mode** on to restock (add products, load coins) and off afterwards. Restocking is allowed only in service mode. Buying isn't allowed in service mode. Service mode can't start in the middle of a purchase.
R8	A stock report lists every slot and the coin stock. The machine starts empty, with no coins, in normal (not service) mode.

Non-functional requirements

NF1	New modes with their own rules (for example an OUT_OF_ORDER mode) should be easy to add.
NF2	The way change is worked out should be replaceable without touching the rest of the machine.

Out of scope: card and UPI payments, buying several items at once, several customers at the same time.

Commands and exact output

Command	Output
`ADD_PRODUCT slot name price qty`	`Added Lays to A1 (Rs 20 x 2)` (replaces whatever was in the slot)
`LOAD_COINS denomination count`	`Loaded coins: Rs 5 x 2`
`SELECT slot`	`Selected Lays, price Rs 20`
`INSERT amount`	not enough yet: `Inserted Rs 10, remaining Rs 10` done, with change: `Dispensed Lays, change Rs 10 [10]` (the coins, largest first, in Java's list format) done, exact money: `Dispensed Lays, no change`
`CANCEL`	`Cancelled, refunded Rs 10`
`SERVICE_ON / SERVICE_OFF`	`Service mode on` / `Service mode off`
`STOCK`	One line per slot, in slot order: `A1 Lays Rs 20 x 1`, then `Coins: 1x4 2x0 5x1 10x2 20x2` (smallest denomination first)

Error	Output
`SELECT an empty slot code`	`No product in slot A1`
`SELECT a product with quantity 0`	`Lays is sold out`
`SELECT during a purchase`	`Finish or cancel the current purchase first`
`INSERT or LOAD_COINS a bad denomination`	`Rejected Rs 3: not accepted`
`INSERT before SELECT`	`Select a product first, returned Rs 10`
`exact change impossible`	`Cannot give change for Rs 3, refunded Rs 20`
`CANCEL with nothing selected`	`Nothing to cancel`
`SELECT, INSERT or CANCEL in service mode`	`Machine is in service mode`
`SERVICE_ON during a purchase`	`Cannot enter service mode during a purchase`
`SERVICE_ON when already in service mode`	`Already in service mode`
`SERVICE_OFF when not in service mode`	`Machine is not in service mode`
`ADD_PRODUCT or LOAD_COINS outside service mode`	`Restocking needs service mode`

Which error wins? The mode is checked first. So in service mode, any buying command gives "Machine is in service mode"; during a purchase, SELECT gives "Finish or cancel..." whatever the slot. For INSERT, the denomination is checked before "Select a product first". For LOAD_COINS, "Restocking needs service mode" comes before the denomination check.

Part A — design

1	Draw a table: one row per mode, one column per operation. Fill every cell with what happens.
2	Which pattern does that table suggest? Why is it worth it here, and when would an enum and a switch be enough?
3	Where does the change-making logic live, and how do you make it replaceable (NF2)?
4	R5: in what order do you do things on the final coin, so that a failed change leaves no trace?

Part B — code

input.txt

```
SELECT A1
ADD_PRODUCT A1 Lays 20 2
SERVICE_ON
ADD_PRODUCT A1 Lays 20 2
ADD_PRODUCT A2 Water 15 1
ADD_PRODUCT B1 Chikki 17 3
LOAD_COINS 5 2
LOAD_COINS 2 1
LOAD_COINS 3 4
SELECT A1
SERVICE_OFF
INSERT 10
SELECT A1
SELECT A2
INSERT 3
INSERT 10
INSERT 20
SELECT A2
INSERT 20
SELECT A2
SELECT B1
INSERT 20
SERVICE_ON
LOAD_COINS 1 5
SERVICE_OFF
SELECT B1
INSERT 10
CANCEL
CANCEL
SELECT B1
INSERT 10
INSERT 10
SELECT A1
SERVICE_ON
CANCEL
SERVICE_OFF
STOCK
```

expected output

```
No product in slot A1
Restocking needs service mode
Service mode on
Added Lays to A1 (Rs 20 x 2)
Added Water to A2 (Rs 15 x 1)
Added Chikki to B1 (Rs 17 x 3)
Loaded coins: Rs 5 x 2
Loaded coins: Rs 2 x 1
Rejected Rs 3: not accepted
Machine is in service mode
Service mode off
Select a product first, returned Rs 10
Selected Lays, price Rs 20
Finish or cancel the current purchase first
Rejected Rs 3: not accepted
Inserted Rs 10, remaining Rs 10
Dispensed Lays, change Rs 10 [10]
Selected Water, price Rs 15
Dispensed Water, change Rs 5 [5]
Water is sold out
Selected Chikki, price Rs 17
Cannot give change for Rs 3, refunded Rs 20
Service mode on
Loaded coins: Rs 1 x 5
Service mode off
Selected Chikki, price Rs 17
Inserted Rs 10, remaining Rs 7
Cancelled, refunded Rs 10
Nothing to cancel
Selected Chikki, price Rs 17
Inserted Rs 10, remaining Rs 7
Dispensed Chikki, change Rs 3 [2, 1]
Selected Lays, price Rs 20
Cannot enter service mode during a purchase
Cancelled, refunded Rs 0
Machine is not in service mode
A1 Lays Rs 20 x 1
A2 Water Rs 15 x 0
B1 Chikki Rs 17 x 2
Coins: 1x4 2x0 5x1 10x2 20x2
```

Part C — pressure questions

1	**Change.** "Add OUT_OF_ORDER: when the motor jams, the machine refuses everything except service mode." What changes in your design?
2	**Change.** Largest-first can fail. Stock: one Rs 5 coin and three Rs 2 coins, change due Rs 6. Largest-first takes the 5, then needs a 1, and fails, though 2 + 2 + 2 works. How would you fix it, and what in your design makes that a small change?
3	**Failure.** "The motor jams after the money is taken." What must the machine do, and in what order?
4	**Defend it.** "Three states with if-else would be shorter than all these classes."

Problem 3 — Library Management

Core · 75 minutes (20 design + 55 code) · machine-coding format

The story. Volunteers run the Pustak Community Library in Deccan. Members borrow and return books at one desk, pay fines for late returns, and join a waiting list when every copy of a popular book is out.

Functional requirements

R1	A book has an ISBN, a title, an author (a single word, the surname) and one or more copies. Copy IDs are `<isbn>-1`, `<isbn>-2`, ...
R2	A member has an ID, a name and a membership type. **BASIC**: up to 2 books at a time, 14-day loans, Rs 10 per day late. **PREMIUM**: up to 5 books, 28-day loans, Rs 5 per day late.
R3	A member borrows by ISBN, and the library hands over a copy. Refused (checked in this order) if: the member is unknown, the book is unknown, the member has an unpaid fine, the member is at their limit, or no copy is available.
R4	Days are plain numbers (day 1, day 2, ...). Due day = borrow day + loan days.
R5	A copy is returned by its copy ID on a given day. If it's late, the fine = days late × the member's rate, added to their unpaid fine.
R6	A member can pay any amount. The library takes only what's owed.
R7	If no copy of a book is available, a member can join its waiting list (first come, first served). Refused if the member is already on that list, or if a copy is available right now (checked in that order, after the member and book exist).
R8	When a copy of a book with a waiting list is returned, it's **held** for the first member on the list, and that member is notified. Only that member can borrow a held copy.
R9	Search finds books whose title or author contains a keyword, ignoring case. Results are in ISBN order, with availability. Held copies don't count as available.
R10	A member summary shows their borrowed copies, in the order borrowed, and their unpaid fine.

Non-functional requirements

NF1	Adding a membership type with different limits should be a one-line change.
NF2	Notifications must not be tangled into the borrow and return logic. Other channels (SMS, email) may be added later.
NF3	A notification line is printed **after** the result line of the command that caused it.

Out of scope: hold expiry, renewals, lost books, several branches, several desks at the same time.

Commands and exact output

Command	Output
`ADD_BOOK isbn copies author title…`	`Added 'Effective Java' by Bloch, copies 101-1 101-2` (the title is the rest of the line)
`ADD_MEMBER id type name`	`Member M1 Asha (BASIC)`
`BORROW member isbn day`	`M1 borrowed 101-1 'Effective Java', due day 15`
`RETURN copyId day`	`101-1 returned on time` or `102-1 returned 3 days late, fine Rs 30` ("1 day", otherwise "days")
`RESERVE member isbn`	`M2 reserved 'Clean Code' (position 1)`
`PAY member amount`	`M1 paid Rs 30, remaining fine Rs 0`
`SEARCH keyword`	one line per match: `101 'Effective Java' by Bloch, 1 of 2 available`, or `No books match 'python'`
`MEMBER id`	`M1 Asha (BASIC): 101-1 (due day 15), 102-1 (due day 16), fine Rs 0`, or `M1 Asha (BASIC): no books, fine Rs 0`
`(notification)`	`Notify M2: 'Clean Code' is ready for pickup`

Error	Output
`unknown member`	`No member M9`
`unknown book`	`No book 999`
`unpaid fine`	`M1 has an unpaid fine of Rs 30`
`at the limit`	`M1 has reached the limit of 2 books`
`no copy available`	`No copy of 'Clean Code' available`
`RETURN a copy that isn't out`	`101-1 is not borrowed`
`RESERVE twice`	`M2 has already reserved 'Clean Code'`
`RESERVE when a copy is available`	`A copy of 'Effective Java' is available, borrow it instead`
`unknown membership type`	`Unknown membership type GOLD`

Part A — design

1	List the entities, and say for each: class, enum, record or field. Are the membership types a Strategy or an enum? Why?
2	Which object owns each rule: "is the member at their limit?", "which copy can this member have?", "how much is the fine?"
3	How does a notification leave the system without the return logic knowing who's listening (NF2)? And how do you print it after the result line (NF3)?
4	What states can a copy be in? Do you need the State pattern for them?

Part B — code

input.txt

```
ADD_BOOK 101 2 Bloch Effective Java
ADD_BOOK 102 1 Martin Clean Code
ADD_BOOK 103 1 Kleppmann Designing Data-Intensive Applications
ADD_MEMBER M1 BASIC Asha
ADD_MEMBER M2 PREMIUM Ravi
ADD_MEMBER M3 BASIC Meera
BORROW M1 101 1
BORROW M1 102 2
BORROW M1 103 3
BORROW M2 102 3
RESERVE M2 102
RESERVE M3 102
RESERVE M2 102
RESERVE M2 101
SEARCH java
SEARCH MARTIN
SEARCH python
MEMBER M1
RETURN 101-1 15
RETURN 102-1 19
BORROW M1 101 20
PAY M1 50
BORROW M1 102 20
BORROW M2 102 21
RETURN 101-1 22
BORROW M9 101 22
BORROW M1 999 22
MEMBER M2
RETURN 102-1 50
MEMBER M1
```

expected output

```
Added 'Effective Java' by Bloch, copies 101-1 101-2
Added 'Clean Code' by Martin, copies 102-1
Added 'Designing Data-Intensive Applications' by Kleppmann, copies 103-1
Member M1 Asha (BASIC)
Member M2 Ravi (PREMIUM)
Member M3 Meera (BASIC)
M1 borrowed 101-1 'Effective Java', due day 15
M1 borrowed 102-1 'Clean Code', due day 16
M1 has reached the limit of 2 books
No copy of 'Clean Code' available
M2 reserved 'Clean Code' (position 1)
M3 reserved 'Clean Code' (position 2)
M2 has already reserved 'Clean Code'
A copy of 'Effective Java' is available, borrow it instead
101 'Effective Java' by Bloch, 1 of 2 available
102 'Clean Code' by Martin, 0 of 1 available
No books match 'python'
M1 Asha (BASIC): 101-1 (due day 15), 102-1 (due day 16), fine Rs 0
101-1 returned on time
102-1 returned 3 days late, fine Rs 30
Notify M2: 'Clean Code' is ready for pickup
M1 has an unpaid fine of Rs 30
M1 paid Rs 30, remaining fine Rs 0
No copy of 'Clean Code' available
M2 borrowed 102-1 'Clean Code', due day 49
101-1 is not borrowed
No member M9
No book 999
M2 Ravi (PREMIUM): 102-1 (due day 49), fine Rs 0
102-1 returned 1 day late, fine Rs 5
Notify M3: 'Clean Code' is ready for pickup
M1 Asha (BASIC): no books, fine Rs 0
```

Part C — pressure questions

1	**Change.** "Add STUDENT: 3 books, 21 days, Rs 2 per day." Then: "Students pay no fines during exam week." What changes each time?
2	**Change.** "A held copy is released if it isn't picked up within 3 days." How do you build that?
3	**Concurrency.** "We open a second desk. Two members borrow the last copy at the same instant." What breaks? What fixes it on one server, and on three?
4	**Defend it.** "Why not `member.borrow(book)` on the `Member` class instead of a service?"

Problem 4 — Parking Lot

Heavy · 100 minutes (25 design + 75 code) · machine-coding format, with a concurrency test

The story. Phoenix Mall in Viman Nagar has a multi-floor car park. Several entry gates issue tickets at the same time. A display board at the entrance shows free spots, and the mall charges more at weekends.

Functional requirements

R1	The lot has F floors. Every floor has the same number of car, bike and truck spots. Spot IDs look like `F1-C1`, `F1-B2`, `F2-T1`: floor, type letter, number.
R2	Each vehicle type parks only in its own spot type.
R3	Parking issues a ticket (`T1`, `T2`, … in order; failed attempts don't use up a number) and assigns a spot using the current **allocation policy**. **LOWEST_FLOOR** (the default): the lowest floor with a free spot, then the lowest spot number. **MOST_FREE_FLOOR**: the floor with the most free spots of that type (a tie goes to the lower floor), then the lowest spot number. The operator can switch policy at any time.
R4	A vehicle that's already parked can't park again.
R5	At exit, fee = hourly rate × hours, where hours = the duration rounded up, with a minimum of 1. Rates: CAR Rs 20, BIKE Rs 10, TRUCK Rs 50. At a weekend the fee is 150% of that, calculated as `fee * 150 / 100` with integer division. The weekend setting at the moment of exit applies.
R6	Times are `HH:MM`, all on the same day. An exit time before the entry time is rejected.
R7	A display board shows the free spots per floor and type. It also announces `<TYPE> FULL` when the last free spot of a type in the whole lot is taken, and `<TYPE> AVAILABLE` when one frees up again.
R8	Display announcements are printed **after** the result line of the command that caused them.

Non-functional requirements

NF1	Several gates park vehicles at the same time. Two vehicles must never get the same spot, and the same vehicle must never be parked twice. (The RUSH command tests this.)
NF2	New allocation policies and new pricing rules (surcharges, discounts) must not require editing the parking logic.
NF3	Parking must not depend on the display. Adding a second display or an analytics feed must not change the park and unpark code.

Out of scope: payment, reservations, stays across midnight, bikes in car spots, EV charging.

Commands and exact output

Command	Output
`CREATE_LOT floors cars bikes trucks`	`Created lot: 2 floors, each with 2 car, 2 bike, 1 truck spots`
`PARK type reg HH:MM`	`Ticket T1: CAR MH12AA1111 at F1-C1, in 09:00`
`UNPARK ticket HH:MM`	`T1: CAR MH12AA1111 left F1-C1 after 3 h, fee Rs 60`
`POLICY name`	`Policy: MOST_FREE_FLOOR`
`DAY WEEKDAY | WEEKEND`	`Day: WEEKEND`
`FREE`	one line per floor: `F1: car 0, bike 1, truck 0`
`RUSH count type HH:MM`	**count** vehicles, with registrations `RUSH-1` … `RUSH-n`, try to park at the same moment from separate threads. Print only `Rush: 2 parked, 48 turned away`, with no ticket lines.
`(announcement)`	`Display: TRUCK FULL`, `Display: CAR AVAILABLE`

Error	Output
`any command before CREATE_LOT`	`No parking lot created`
`unknown vehicle type`	`Unknown vehicle type BUS`
`vehicle already parked`	`MH12AA1111 is already parked`
`no free spot of that type`	`No TRUCK spot available for MH12TR9999`
`unknown or already used ticket`	`Invalid ticket T3`
`exit before entry`	`Exit time 08:30 is before entry 09:00`
`unknown policy / unknown day`	`Unknown policy FASTEST` / `Unknown day HOLIDAY`

Part A — design

1	Entities and relationships, with multiplicity. Draw the class diagram.
2	Choose a design for each: the allocation policy chosen by name; weekend pricing on top of hourly pricing; the display board reacting; the different rates per vehicle type. Give the reason for each, and say where you deliberately used no pattern.
3	NF1: exactly which steps must happen as one indivisible step, and what do you lock? What must **not** be inside the lock?
4	Write the order of steps inside park and unpark. Mark each one: required, or announcement.

Part B — code

input.txt

```
PARK CAR MH12AA1111 09:00
CREATE_LOT 2 2 2 1
PARK CAR MH12AA1111 09:00
PARK CAR MH12BB2222 09:15
PARK BIKE MH12CC3333 09:20
PARK CAR MH12AA1111 09:30
POLICY MOST_FREE_FLOOR
PARK CAR MH14DD4444 09:40
PARK BIKE MH12EE5555 09:45
POLICY LOWEST_FLOOR
PARK BIKE MH12FF6666 09:50
PARK TRUCK MH12TR7777 10:00
PARK TRUCK MH12TR8888 10:05
PARK TRUCK MH12TR9999 10:10
PARK BUS MH12BU0000 10:15
FREE
UNPARK T1 08:30
UNPARK T1 11:30
DAY WEEKEND
UNPARK T3 10:20
UNPARK T3 10:30
UNPARK T8 12:00
DAY WEEKDAY
POLICY FASTEST
RUSH 50 CAR 13:00
FREE
PARK CAR MH12GG7777 13:30
UNPARK T2 14:00
PARK CAR MH12GG7777 14:05
```

expected output

```
No parking lot created
Created lot: 2 floors, each with 2 car, 2 bike, 1 truck spots
Ticket T1: CAR MH12AA1111 at F1-C1, in 09:00
Ticket T2: CAR MH12BB2222 at F1-C2, in 09:15
Ticket T3: BIKE MH12CC3333 at F1-B1, in 09:20
MH12AA1111 is already parked
Policy: MOST_FREE_FLOOR
Ticket T4: CAR MH14DD4444 at F2-C1, in 09:40
Ticket T5: BIKE MH12EE5555 at F2-B1, in 09:45
Policy: LOWEST_FLOOR
Ticket T6: BIKE MH12FF6666 at F1-B2, in 09:50
Ticket T7: TRUCK MH12TR7777 at F1-T1, in 10:00
Ticket T8: TRUCK MH12TR8888 at F2-T1, in 10:05
Display: TRUCK FULL
No TRUCK spot available for MH12TR9999
Unknown vehicle type BUS
F1: car 0, bike 0, truck 0
F2: car 1, bike 1, truck 0
Exit time 08:30 is before entry 09:00
T1: CAR MH12AA1111 left F1-C1 after 3 h, fee Rs 60
Day: WEEKEND
T3: BIKE MH12CC3333 left F1-B1 after 1 h, fee Rs 15
Invalid ticket T3
T8: TRUCK MH12TR8888 left F2-T1 after 2 h, fee Rs 150
Display: TRUCK AVAILABLE
Day: WEEKDAY
Unknown policy FASTEST
Rush: 2 parked, 48 turned away
Display: CAR FULL
F1: car 0, bike 1, truck 0
F2: car 0, bike 1, truck 1
No CAR spot available for MH12GG7777
T2: CAR MH12BB2222 left F1-C2 after 5 h, fee Rs 100
Display: CAR AVAILABLE
Ticket T11: CAR MH12GG7777 at F1-C2, in 14:05
Display: CAR FULL
```

> **About the RUSH line.** If your locking is wrong, the output will *sometimes* show more cars parked than there are spots, a ticket number out of sequence afterwards, or the FULL line twice. "Sometimes" is the key word: a race may hide for many runs. Run the test several times, but trust your Part A reasoning more than a lucky run.

Part C — pressure questions

1	**Change.** "Bikes may take a free car spot when the bike spots are full. They still pay the bike rate." What changes?
2	**Change.** "Add EV charging spots, with a Rs 15 per hour charging fee on top of parking." Where does that go?
3	**Scale.** "The mall now runs three servers behind a load balancer." What happens to your locking, and what replaces it?
4	**Failure.** "Payment fails at the exit barrier." What must not happen, and what should?
5	**Defend it.** "Why an Observer for one display board? Just update it inside `park()`."

Hints

Read one level at a time, and only after 15 minutes of being stuck. Each level gives away more than the last.

Problem 1 — Tic-Tac-Toe

Hint 1 (a nudge). Separate "where things are" from "what the rules are". One class shouldn't do both.

Hint 2 (the shape). `Board` (the grid: inside?, empty?, full?, place, render) · `Game` (players, whose turn, status, `move`) · `Player` as a record · `Symbol` and `GameStatus` as enums. No design pattern is needed.

Hint 3 (the key decision). A winning line must pass through the cell just played. So check only that cell's row, its column, and the two diagonals if the cell is on them. That's n cells per line, not n × n.

Problem 2 — Vending Machine

Hint 1. Write the table first: three modes (idle, selected, service) × six operations (select, insert, cancel, service on, service off, restock). Count the cells.

Hint 2. State pattern: one class per mode, all implementing one interface, holding no data of their own. The machine holds the data (inventory, coin stock, the selected slot, the coins inserted so far) and passes each call to its current state. Change-making sits behind a small interface of its own.

Hint 3. On the coin that completes the price: copy the coin stock, add the inserted coins to the *copy*, and try to make change from the copy. Only if that works, commit to the real stock and reduce the quantity. Otherwise refund, go back to idle, and *then* throw.

Problem 3 — Library Management

Hint 1. The membership types differ only in their numbers. A copy has only a few statuses, and only borrowing and returning care about them.

Hint 2. `Book` (its copies and its waiting list) · `BookCopy` (status, borrower, held for, due day) · `Member` (borrowed copies, fine, `atLimit()`) · `MembershipType` as an enum with fields · `LibraryService` as the one front door · a listener interface for "a copy is ready".

Hint 3. Let the listener collect messages in a list (an "outbox"). The driver prints the command's result, then prints and clears the outbox. In a return: required work first (fine, free the copy), then hold it for the next person in the queue, then announce.

Problem 4 — Parking Lot

Hint 1. Three things vary: which spot (a policy picked by name), what to charge (an hourly fee, sometimes with a weekend layer on top), and who reacts (the display). The vehicle types differ only in numbers.

Hint 2. `AllocationPolicy` (Strategy) built by a small factory from its name · `FeeCalculator` with `HourlyFee` and a `WeekendSurcharge` that wraps it (Decorator) · `ParkingListener` with a `DisplayBoard` (Observer) · `VehicleType` as an enum with a rate · `ParkingService` as the front door.

Hint 3. Inside one `synchronized` block: the "already parked?" check, choosing the spot, occupying it, and creating the ticket. After the block, outside the lock, notify the listeners. The display board collects its announcements, and the driver prints them after the result line.

Solution 1 — Tic-Tac-Toe

Questions a strong candidate asks first

Question	Answer assumed here
Only N × N, or other shapes?	Square, N ≥ 3
Is the win rule always a full row, column or diagonal?	For now, yes; 5 in a row is coming (NF2)
Human players only, or a computer player?	Two humans
What happens on a bad move?	Rejected; the same player tries again
Can a new game start in the middle of one?	Yes; it replaces the old one

Part A — model answer

Entity	Kind	Why
`Symbol`	enum (X, O)	a fixed set of two
`Player`	record	a name and a symbol that never change
`Board`	class	the grid and its geometry: inside, empty, full, place, render
`Game`	class	the rules: whose turn, what's allowed, when it ends
`GameStatus`	enum	IN_PROGRESS, WON, DRAW. Only `move()` checks it, so no State pattern is needed
`WinChecker`	class	the win rule kept in one place, so it can become a Strategy when a second rule arrives (NF2)

Question	Owner	Why
Is this cell empty?	`Board`	it holds the cells
Whose turn is it?	`Game`	it holds the players and the turn
Did that move win?	`WinChecker`	a rule about the board; kept apart so it can be swapped

```
Game 1 ◆——— 1 Board          Game 1 ———> 2 Player ———> Symbol          Game ———> WinChecker
```

Patterns: none, on purpose. Nothing here varies yet. There is one win rule, one kind of player and one board shape. The only thing worth preparing is a **seam**: the win check lives in its own class, so when a second rule arrives, it becomes an interface (Strategy) with one change. Building a Strategy for a single rule today would be YAGNI.

NF1: check only the lines through the last move. A move can only complete a line it belongs to: its row, its column, and a diagonal if it sits on one. That's at most 4 × n cells, instead of n × n. Note the guard: a cell is on the main diagonal only if `row == col`, and on the other diagonal only if `row + col == n - 1`.

Part B — the code

One file, so you can run it in one go. In a real project, each class would get its own file.

```
import java.util.*;

enum Symbol { X, O }

enum GameStatus { IN_PROGRESS, WON, DRAW }

record Player(String name, Symbol symbol) {}

class InvalidMoveException extends RuntimeException {
    InvalidMoveException(String message) { super(message); }
}

// The grid only: where things are. No rules about turns or winning.
class Board {
    private final int size;
    private final Symbol[][] cells;
    private int filled = 0;

    Board(int size) {
        this.size = size;
        this.cells = new Symbol[size][size];       // null means empty
    }

    int size()                      { return size; }
    boolean isInside(int r, int c)  { return r >= 0 && r < size && c >= 0 && c < size; }
    boolean isEmpty(int r, int c)   { return cells[r][c] == null; }
    boolean isFull()                { return filled == size * size; }
    Symbol at(int r, int c)         { return cells[r][c]; }

    void place(int r, int c, Symbol symbol) {
        cells[r][c] = symbol;
        filled++;
    }

    String render() {
        StringBuilder sb = new StringBuilder();
        for (int r = 0; r < size; r++) {
            for (int c = 0; c < size; c++) {
```

```
                if (c > 0) sb.append(' ');
                sb.append(cells[r][c] == null ? "." : cells[r][c].name());
            }
            sb.append('\n');
        }
        return sb.toString();
    }
}

// Checks only the lines that pass through the last move: O(n), not O(n*n).
class WinChecker {
    boolean isWinningMove(Board b, int row, int col, Symbol s) {
        int n = b.size();
        boolean rowWin = true, colWin = true;
        boolean diagWin = (row == col), antiWin = (row + col == n - 1);
        for (int i = 0; i < n; i++) {
            if (b.at(row, i) != s) rowWin = false;
            if (b.at(i, col) != s) colWin = false;
            if (diagWin && b.at(i, i) != s) diagWin = false;
            if (antiWin && b.at(i, n - 1 - i) != s) antiWin = false;
        }
        return rowWin || colWin || diagWin || antiWin;
    }
}

// The rules: whose turn it is, what is allowed, when the game ends.
class Game {
    private final Board board;
    private final Player[] players;
    private final WinChecker winChecker = new WinChecker();
    private int turn = 0;
    private GameStatus status = GameStatus.IN_PROGRESS;

    Game(int size, Player first, Player second) {
        if (size < 3) throw new InvalidMoveException("Board size must be at least 3");
        this.board = new Board(size);
        this.players = new Player[] { first, second };
    }

    // row and col are 1-based, as the players type them
    Player move(int row, int col) {
        if (status != GameStatus.IN_PROGRESS) throw new InvalidMoveException("Game is over");
        int r = row - 1, c = col - 1;
        if (!board.isInside(r, c))
            throw new InvalidMoveException("Invalid move: (" + row + "," + col + ") is outside the board");
        if (!board.isEmpty(r, c))
            throw new InvalidMoveException("Invalid move: (" + row + "," + col + ") is already taken");

        Player current = players[turn];
        board.place(r, c, current.symbol());

        if (winChecker.isWinningMove(board, r, c, current.symbol())) status = GameStatus.WON;
        else if (board.isFull())                                     status = GameStatus.DRAW;
        else                                                         turn = 1 - turn;
        return current;
    }

    GameStatus status() { return status; }
    Board board()       { return board; }
}

public class Main {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        Game game = null;
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                switch (p[0]) {
                    case "START" -> {
                        int n = Integer.parseInt(p[1]);
                        Player a = new Player(p[2], Symbol.X);
                        Player b = new Player(p[3], Symbol.O);
                        game = new Game(n, a, b);
                        System.out.println("Game started: " + n + "x" + n + ", "
                                + a.name() + " (X) vs " + b.name() + " (O)");
                    }
                    case "MOVE" -> {
                        if (game == null) throw new InvalidMoveException("No game in progress");
                        int row = Integer.parseInt(p[1]), col = Integer.parseInt(p[2]);
                        Player mover = game.move(row, col);
                        System.out.println(mover.name() + " placed " + mover.symbol()
                                + " at (" + row + "," + col + ")");
                        if (game.status() == GameStatus.WON)  System.out.println(mover.name() + " wins!");
                        if (game.status() == GameStatus.DRAW) System.out.println("It's a draw!");
                    }
                    case "PRINT" -> {
                        if (game == null) throw new InvalidMoveException("No game in progress");
                        System.out.print(game.board().render());
                    }
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (InvalidMoveException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

How to read it: `Board` knows only geometry. `Game.move()` does the checks in the order the requirements give: over, inside, empty. Then it places, then decides the status, and switches the turn only if the game goes on. The driver prints; nothing else does.

Part C — model answers

1. Undo. Store each successful move as a small object, `Move(row, col, player)`, on a stack. Push it **after** the move succeeds (the Command lesson: only record what actually happened). Undo pops it and must restore **three** things: clear the cell (and decrement the filled count), give the turn back to that player, and set the status back to IN_PROGRESS, because a winning move can be undone too. Command, or Memento-style snapshots, both work; one move is tiny, so Command is the lighter choice.

2. Five in a row on 10 × 10. Turn `WinChecker` into an interface, `WinRule`, with two implementations: `FullLineRule` (today's) and `KInARowRule(k)`, which counts matching symbols outward from the last move in four directions (across, down, both diagonals). `Game` receives the rule in its constructor. `Board` doesn't change at all, and neither does the turn logic.

3. Constant time per move. Keep counters: one per row, one per column, and one for each diagonal. X adds +1 and O adds −1 to the counters its move touches. A line is won the moment its counter reaches +n or −n, because cells are never overwritten, so the counter can only reach ±n when all n cells hold the same symbol. That's four updates and four checks per move, however big the board.

4. "Why separate Board and Game?" "You'd be right for a 50-line script; one class would be fine. I split them because they change for different reasons: the board changes if the shape changes, the game changes if the rules change. The same board could serve Connect Four or Gomoku with a different `Game`, and each can be tested on its own."

Common mistakes
- Scanning the whole board after every move (NF1).
- Checking a diagonal for a cell that isn't on it: forgetting the `row == col` guard.
- Switching the turn after an **invalid** move, or after the **winning** move (then the next prompt names the wrong player).
- Mixing up 0-based arrays with the 1-based input: off by one on every move.
- Returning an error string from `move()` instead of throwing.
- Exact text: `(1,1)` has no space after the comma.

Score yourself (1 point each)

☐	Board and Game are separate classes with clear jobs
☐	Player is a record; Symbol and GameStatus are enums
☐	No pattern was forced in, and you can say why
☐	The win check looks only at the lines through the last move
☐	The checks in `move()` are in the right order (over, inside, empty)
☐	Invalid moves throw, and only the driver prints
☐	The turn switches only after a valid move that doesn't end the game
☐	Your output matches the expected output exactly
☐	Part C 1: undo restores the cell, the turn and the status
☐	Part C 4: you opened with a concession

Solution 2 — Vending Machine

Questions a strong candidate asks first

Question	Answer assumed here
Select first, or insert money first?	Select first, then insert
Which coins and notes are accepted?	Rs 1, 2, 5, 10, 20
How is change chosen, and what if it can't be made?	Largest first; if impossible, refund everything
Who restocks, and when?	An operator, only in service mode
Can two customers use it at once?	No: one customer at a time

Part A — model answer

1. The state table. Every cell is a rule. That's 18 rules, and they're the whole design.

	select	insert	cancel	service on	service off	restock
IDLE	check slot and stock → SELECTED	reject bad coin, else "select first"	"Nothing to cancel"	→ SERVICE	"not in service"	"needs service mode"
SELECTED	"finish or cancel first"	check coin, add it, maybe complete	refund → IDLE	"during a purchase"	"not in service"	"needs service mode"
SERVICE	"in service mode"	"in service mode"	"in service mode"	"already"	→ IDLE	allowed

2. Pattern: State. Many modes × many operations, with rules in every cell, is exactly the signal. Each state class is one row of the table, so reading the code is reading the table. Adding OUT_OF_ORDER (NF1) is one new class. An enum and a `switch` would be enough for two modes and two operations; at 3 × 6 and growing, every method would carry the same if-chain.
The states hold no data. They're shared constants (`MachineState.IDLE` and so on). The machine (the *context*) holds the inventory, the coin stock, the selected slot and the inserted coins, and passes each call to its current state.

3. Change-making: Strategy (NF2). `ChangeStrategy.makeChange(amount, stock)` returns `Optional<List<Integer>>`: the coins, or empty if it's impossible. `Optional` fits because "maybe there's an answer" is a normal outcome here. The machine receives the strategy in its constructor, so swapping it is one line in `main`.

4. R5: check before you change. On the final coin: (a) copy the coin stock; (b) add the inserted coins to the copy; (c) try to make change from the copy. If it fails: refund, reset to IDLE, *then* throw, so nothing changed. If it works: add the inserted coins to the real stock, remove the change, reduce the quantity, reset to IDLE.

Entity	Kind	Job
`VendingMachine`	class (context)	holds the data; delegates every operation to the current state
`MachineState` + 3 states	interface + classes	the rules, one class per row of the table
`Inventory`, `Slot`	classes	slots in order; each slot's product and quantity
`Product`	record	name and price, never changes
`CoinStock`	class	counts per denomination; can copy itself for a trial run
`ChangeStrategy`, `GreedyChange`	interface + class	how change is worked out
`InsertOutcome`	record	the result of an insert: dispensed or not, remaining, change

Deliberately not used: Observer (nobody needs to react yet), Factory (nothing to choose from text), a Singleton class (one machine is simply created once in `main`).

Part B — the code

```
import java.util.*;

class VendingException extends RuntimeException {
    VendingException(String message) { super(message); }
}

record Product(String name, int price) {}

class Slot {
    final String code;
    final Product product;
    int quantity;
    Slot(String code, Product product, int quantity) {
        this.code = code; this.product = product; this.quantity = quantity;
    }
}

class Inventory {
    private final Map<String, Slot> slots = new TreeMap<>();      // sorted by slot code

    void put(String code, Product product, int quantity) {
        slots.put(code, new Slot(code, product, quantity));
    }

    Slot get(String code) {
        Slot slot = slots.get(code);
```

```
        if (slot == null) throw new VendingException("No product in slot " + code);
        return slot;
    }

    Collection<Slot> all() { return slots.values(); }
}

class CoinStock {
    static final int[] DENOMINATIONS = {20, 10, 5, 2, 1};          // largest first
    private final Map<Integer, Integer> counts = new TreeMap<>();

    CoinStock() { for (int d : DENOMINATIONS) counts.put(d, 0); }

    static boolean accepts(int amount) { return Arrays.stream(DENOMINATIONS).anyMatch(d -> d == amount); }

    int count(int d)                    { return counts.get(d); }
    void add(int d, int howMany)        { counts.merge(d, howMany, Integer::sum); }
    void addAll(List<Integer> coins)    { for (int c : coins) add(c, 1); }
    void removeAll(List<Integer> coins) { for (int c : coins) add(c, -1); }

    CoinStock copy() {
        CoinStock c = new CoinStock();
        c.counts.putAll(this.counts);
        return c;
    }

    String describe() {                                             // ascending: 1x4 2x0 ...
        StringJoiner j = new StringJoiner(" ");
        counts.forEach((d, n) -> j.add(d + "x" + n));
        return j.toString();
    }
}

interface ChangeStrategy {
    Optional<List<Integer>> makeChange(int amount, CoinStock stock);   // empty = impossible
}

class GreedyChange implements ChangeStrategy {
    public Optional<List<Integer>> makeChange(int amount, CoinStock stock) {
        List<Integer> coins = new ArrayList<>();
        int remaining = amount;
        for (int d : CoinStock.DENOMINATIONS) {
            int use = Math.min(stock.count(d), remaining / d);
            for (int i = 0; i < use; i++) coins.add(d);
            remaining -= use * d;
        }
        return remaining == 0 ? Optional.of(coins) : Optional.empty();
    }
}

record InsertOutcome(Product dispensed, int remaining, List<Integer> change) {
    boolean completed() { return dispensed != null; }
}

// ---------------- the states ----------------
interface MachineState {
    Product select(VendingMachine m, String code);
    InsertOutcome insert(VendingMachine m, int amount);
    int cancel(VendingMachine m);
    void serviceOn(VendingMachine m);
    void serviceOff(VendingMachine m);
    void checkCanRestock();

    MachineState IDLE = new IdleState();
    MachineState SELECTED = new SelectedState();
    MachineState SERVICE = new ServiceState();
}

class IdleState implements MachineState {
    public Product select(VendingMachine m, String code) {
        Slot slot = m.inventory().get(code);
        if (slot.quantity == 0) throw new VendingException(slot.product.name() + " is sold out");
        m.startPurchase(slot);
        return slot.product;
    }
    public InsertOutcome insert(VendingMachine m, int amount) {
        if (!CoinStock.accepts(amount))
            throw new VendingException("Rejected Rs " + amount + ": not accepted");
        throw new VendingException("Select a product first, returned Rs " + amount);
    }
    public int cancel(VendingMachine m)      { throw new VendingException("Nothing to cancel"); }
    public void serviceOn(VendingMachine m)  { m.setState(SERVICE); }
    public void serviceOff(VendingMachine m) { throw new VendingException("Machine is not in service mode"); }
    public void checkCanRestock()            { throw new VendingException("Restocking needs service mode"); }
}

class SelectedState implements MachineState {
    public Product select(VendingMachine m, String code) {
        throw new VendingException("Finish or cancel the current purchase first");
    }
    public InsertOutcome insert(VendingMachine m, int amount) {
        if (!CoinStock.accepts(amount))
            throw new VendingException("Rejected Rs " + amount + ": not accepted");
        m.addInserted(amount);
        return m.completeIfPaid();
    }
    public int cancel(VendingMachine m)      { return m.refundAndReset(); }
    public void serviceOn(VendingMachine m) {
        throw new VendingException("Cannot enter service mode during a purchase");
    }
    public void serviceOff(VendingMachine m) { throw new VendingException("Machine is not in service mode"); }
```

```
    public void checkCanRestock()            { throw new VendingException("Restocking needs service mode"); }
}

class ServiceState implements MachineState {
    public Product select(VendingMachine m, String code)      { throw inService(); }
    public InsertOutcome insert(VendingMachine m, int amount) { throw inService(); }
    public int cancel(VendingMachine m)                       { throw inService(); }
    public void serviceOn(VendingMachine m)  { throw new VendingException("Already in service mode"); }
    public void serviceOff(VendingMachine m) { m.setState(IDLE); }
    public void checkCanRestock()            { }                                    // allowed
    private VendingException inService()     { return new VendingException("Machine is in service mode"); }
}

// ---------------- the context: holds the data, delegates the rules ----------------
class VendingMachine {
    private final Inventory inventory = new Inventory();
    private final CoinStock coins = new CoinStock();
    private final ChangeStrategy changeStrategy;
    private MachineState state = MachineState.IDLE;
    private Slot selected;
    private final List<Integer> inserted = new ArrayList<>();      // held until the purchase completes

    VendingMachine(ChangeStrategy changeStrategy) { this.changeStrategy = changeStrategy; }

    // operations: each one asks the current state
    Product select(String code)    { return state.select(this, code); }
    InsertOutcome insert(int amt)  { return state.insert(this, amt); }
    int cancel()                   { return state.cancel(this); }
    void serviceOn()               { state.serviceOn(this); }
    void serviceOff()              { state.serviceOff(this); }

    void addProduct(String code, String name, int price, int qty) {
        state.checkCanRestock();
        inventory.put(code, new Product(name, price), qty);
    }

    void loadCoins(int denomination, int count) {
        state.checkCanRestock();
        if (!CoinStock.accepts(denomination))
            throw new VendingException("Rejected Rs " + denomination + ": not accepted");
        coins.add(denomination, count);
    }

    // helpers the states use
    Inventory inventory()          { return inventory; }
    void setState(MachineState s)  { this.state = s; }
    void startPurchase(Slot slot)  { selected = slot; setState(MachineState.SELECTED); }
    void addInserted(int amount)   { inserted.add(amount); }

    int refundAndReset() {
        int refund = inserted.stream().mapToInt(Integer::intValue).sum();
        inserted.clear();
        selected = null;
        setState(MachineState.IDLE);
        return refund;
    }

    InsertOutcome completeIfPaid() {
        int paid = inserted.stream().mapToInt(Integer::intValue).sum();
        int price = selected.product.price();
        if (paid < price) return new InsertOutcome(null, price - paid, List.of());

        int changeDue = paid - price;
        CoinStock trial = coins.copy();                // check BEFORE changing anything
        trial.addAll(inserted);
        Optional<List<Integer>> change = changeStrategy.makeChange(changeDue, trial);
        if (change.isEmpty()) {
            int refund = refundAndReset();             // clean up first...
            throw new VendingException("Cannot give change for Rs " + changeDue
                    + ", refunded Rs " + refund);
        }

        coins.addAll(inserted);                        // commit: the money joins the stock
        coins.removeAll(change.get());                 // pay the change
        selected.quantity--;
        Product product = selected.product;
        inserted.clear();
        selected = null;
        setState(MachineState.IDLE);
        return new InsertOutcome(product, 0, change.get());
    }

    String stockReport() {
        StringBuilder sb = new StringBuilder();
        for (Slot s : inventory.all())
            sb.append(s.code).append(' ').append(s.product.name())
              .append(" Rs ").append(s.product.price()).append(" x ").append(s.quantity).append('\n');
        sb.append("Coins: ").append(coins.describe()).append('\n');
        return sb.toString();
    }
}

public class Main {
    public static void main(String[] args) {
        VendingMachine machine = new VendingMachine(new GreedyChange());
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
```

```
                switch (p[0]) {
                    case "ADD_PRODUCT" -> {
                        int price = Integer.parseInt(p[3]), qty = Integer.parseInt(p[4]);
                        machine.addProduct(p[1], p[2], price, qty);
                        System.out.println("Added " + p[2] + " to " + p[1]
                                + " (Rs " + price + " x " + qty + ")");
                    }
                    case "LOAD_COINS" -> {
                        int d = Integer.parseInt(p[1]), n = Integer.parseInt(p[2]);
                        machine.loadCoins(d, n);
                        System.out.println("Loaded coins: Rs " + d + " x " + n);
                    }
                    case "SELECT" -> {
                        Product chosen = machine.select(p[1]);
                        System.out.println("Selected " + chosen.name() + ", price Rs " + chosen.price());
                    }
                    case "INSERT" -> {
                        int amount = Integer.parseInt(p[1]);
                        InsertOutcome out = machine.insert(amount);
                        if (!out.completed())
                            System.out.println("Inserted Rs " + amount
                                    + ", remaining Rs " + out.remaining());
                        else if (out.change().isEmpty())
                            System.out.println("Dispensed " + out.dispensed().name() + ", no change");
                        else {
                            int total = out.change().stream().mapToInt(Integer::intValue).sum();
                            System.out.println("Dispensed " + out.dispensed().name()
                                    + ", change Rs " + total + " " + out.change());
                        }
                    }
                    case "CANCEL" -> System.out.println("Cancelled, refunded Rs " + machine.cancel());
                    case "SERVICE_ON" -> { machine.serviceOn();  System.out.println("Service mode on"); }
                    case "SERVICE_OFF" -> { machine.serviceOff(); System.out.println("Service mode off"); }
                    case "STOCK" -> System.out.print(machine.stockReport());
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (VendingException e) {
                System.out.println(e.getMessage());
            }
        }
    }
}
```

How to read it: the interface declares the three state constants, so states can switch with `m.setState(IDLE)`. Each state class reads as one row of the table. All the money logic lives in `completeIfPaid()`, which follows the four steps of answer 4 exactly.

Part C — model answers

1. OUT_OF_ORDER. One new class, `OutOfOrderState`: buying operations throw "Machine is out of order", and `serviceOn()` is allowed so the operator can repair it. Something must move the machine into it (a `jam()` call from the motor). No existing state changes, apart from the one that triggers the jam. This is the trade-off of the pattern: a new **state** is one class, but a new **operation** means a method in every state. The compiler lists them all for you, so none is forgotten.

2. When largest-first fails. Write `ExactChange implements ChangeStrategy`, which tries combinations: backtracking over the denominations and their counts, largest first, stepping back when a path fails. The amounts are tiny, so it's fast. Then change one line in `main`: `new VendingMachine(new ExactChange())`. Nothing else moves. That's NF2 paying off.

3. The motor jams after the money is taken. Order the steps so the money isn't committed until the product is out: check change → dispense → commit the coins and pay change. If dispensing fails: **refund** the inserted coins (compensation), leave the quantity alone, move the machine to OUT_OF_ORDER, **alert** the operator (a listener: SMS or email), and report the failure. Never keep the money for a product that didn't drop.

4. "If-else would be shorter." "You'd be right for two modes and two operations; I'd use a switch there. Here it's three modes by six operations, so 18 rules, and an OUT_OF_ORDER mode is already on the way. With if-else, every method repeats the same mode check, and a new mode means editing all of them. With State, each class is one row of the table, and a new mode is one new class."

Common mistakes

- Adding the inserted coins to the real stock **before** knowing change is possible, so a refund leaves the stock wrong.
- Forgetting to go back to IDLE after a refund: the machine is stuck mid-purchase.
- Storing purchase data (the inserted coins, the selected slot) **inside** a state class. The states are shared and must hold no data.
- An if-chain on the mode inside the machine's methods *as well as* state classes: the rules end up in two places.
- Money as `double`. Rupees are whole numbers: `int`.
- Exact text: the change list is `[2, 1]`, Java's own list format, largest coin first.

Score yourself (1 point each)

☐	You drew the state table before coding
☐	One class per state, and the states hold no data
☐	The machine holds the data and delegates every operation
☐	Change-making sits behind an interface
☐	Change is tried on a copy before anything is committed
☐	On failure: refund and reset first, then throw
☐	The errors come out in the right precedence
☐	Money is `int`
☐	Your output matches exactly
☐	Part C 1: a new mode is one new class, and you named the trade-off

Solution 3 — Library Management

Questions a strong candidate asks first

Question	Answer assumed here
What are the membership types and their limits?	BASIC and PREMIUM, numbers as in R2
How are fines worked out, and does an unpaid fine block borrowing?	Per day late, by type; yes, it blocks
What happens when every copy is out?	A waiting list; the returned copy is held for the first in line
How long does a hold last?	Out of scope for now (Part C 2)
One desk or several at once?	One desk (Part C 3 asks about two)

Part A — model answer

Entity	Kind	Why
`Book`	class	holds its copies and its waiting list; answers "which copy can this member have?"
`BookCopy`	class	a physical copy with a status, a borrower, a due day, a holder
`Member`	class	borrowed copies and fine; answers "am I at my limit?"
`MembershipType`	**enum with fields**	the types differ only in numbers (limit, days, rate), so no Strategy. NF1: a new type is one line
`CopyStatus`	enum	AVAILABLE, BORROWED, ON_HOLD; checked only in borrow and return, so no State pattern
`LibraryService`	class (Facade)	the one front door; coordinates members, books, fines and holds
`LibraryListener`	interface (Observer)	"a copy is ready for this member"; the service doesn't know who listens
`ReturnResult`	record	days late and fine, for the driver to print

Rule	Owner	Why
Is the member at their limit?	`Member.atLimit()`	it holds the borrowed list and the type
Which copy can this member have?	`Book.copyFor(m)`	it holds the copies; a copy held for *m* wins
How much is the fine?	days late × `type.finePerDay`	the numbers live on the type
Who's next for a returned copy?	`Book.waitlist`	a queue: first come, first served

```
LibraryService ———> * Book 1 ◆——— 1..* BookCopy ———> 0..1 Member (borrower or holder)
LibraryService ———> * Member ———> MembershipType          Book 1 ———> * Member (waiting list)
LibraryService ———> * LibraryListener  ◁- - -  NotificationOutbox
```

A copy's life:

```
AVAILABLE ——borrow——> BORROWED ——return, nobody waiting——> AVAILABLE
                        |
                        └——return, someone waiting——> ON_HOLD ——borrow, by the holder only——> BORROWED
```

Notifications (NF2, NF3). The service calls `onCopyReady(member, copy)` on every listener **after** the required work is done. The listener here is an **outbox**: it just collects the message. The driver prints the command's result, then prints and clears the outbox. So the order on screen is "returned…" then "Notify…", and a future SMS or email channel is one more listener, with no change to the service.

The order inside a return: validate the copy → work out and add the fine → take it off the member and mark it available → if someone's waiting, hold it for them → announce. Required steps first, announcement last.

Part B — the code

```
import java.util.*;

class LibraryException extends RuntimeException {
    LibraryException(String message) { super(message); }
```

```
}

// The types differ only in their numbers, so an enum with fields is enough.
enum MembershipType {
    BASIC(2, 14, 10),
    PREMIUM(5, 28, 5);

    final int maxBooks, loanDays, finePerDay;
    MembershipType(int maxBooks, int loanDays, int finePerDay) {
        this.maxBooks = maxBooks; this.loanDays = loanDays; this.finePerDay = finePerDay;
    }

    static MembershipType parse(String s) {
        try { return valueOf(s); }
        catch (IllegalArgumentException e) { throw new LibraryException("Unknown membership type " + s); }
    }
}

enum CopyStatus { AVAILABLE, BORROWED, ON_HOLD }

class Member {
    final String id, name;
    final MembershipType type;
    final List<BookCopy> borrowed = new ArrayList<>();
    int fine = 0;

    Member(String id, String name, MembershipType type) {
        this.id = id; this.name = name; this.type = type;
    }

    boolean atLimit() { return borrowed.size() >= type.maxBooks; }
}

class Book {
    final String isbn, title, author;
    final List<BookCopy> copies = new ArrayList<>();
    final Deque<Member> waitlist = new ArrayDeque<>();            // reservations, first come first served

    Book(String isbn, String title, String author) {
        this.isbn = isbn; this.title = title; this.author = author;
    }

    long availableCount() {
        return copies.stream().filter(c -> c.status == CopyStatus.AVAILABLE).count();
    }

    // A copy held for this member wins; otherwise any available copy.
    Optional<BookCopy> copyFor(Member m) {
        for (BookCopy c : copies)
            if (c.status == CopyStatus.ON_HOLD && c.heldFor == m) return Optional.of(c);
        for (BookCopy c : copies) if (c.status == CopyStatus.AVAILABLE) return Optional.of(c);
        return Optional.empty();
    }

    boolean matches(String keyword) {
        String k = keyword.toLowerCase();
        return title.toLowerCase().contains(k) || author.toLowerCase().contains(k);
    }
}

class BookCopy {
    final String id;
    final Book book;
    CopyStatus status = CopyStatus.AVAILABLE;
    Member borrower;
    Member heldFor;
    int dueDay;

    BookCopy(String id, Book book) { this.id = id; this.book = book; }

    void lendTo(Member m, int dueDay) {
        status = CopyStatus.BORROWED; borrower = m; heldFor = null; this.dueDay = dueDay;
    }
    void returned()                   { status = CopyStatus.AVAILABLE; borrower = null; }
    void holdFor(Member m)            { status = CopyStatus.ON_HOLD; heldFor = m; }
}

record ReturnResult(int daysLate, int fine) {}

interface LibraryListener {
    void onCopyReady(Member member, BookCopy copy);
}

// Collects notifications; the driver prints them AFTER the command's own result.
class NotificationOutbox implements LibraryListener {
    private final List<String> pending = new ArrayList<>();
    public void onCopyReady(Member member, BookCopy copy) {
        pending.add("Notify " + member.id + ": '" + copy.book.title + "' is ready for pickup");
    }
    List<String> drain() {
        List<String> out = new ArrayList<>(pending);
        pending.clear();
        return out;
    }
}

// The front door for every command. In-memory maps stand in for repositories.
class LibraryService {
    private final Map<String, Book> books = new TreeMap<>();         // sorted by ISBN for search
    private final Map<String, Member> members = new HashMap<>();
    private final Map<String, BookCopy> copies = new HashMap<>();
```

```
    private final List<LibraryListener> listeners = new ArrayList<>();

    void subscribe(LibraryListener l) { listeners.add(l); }

    Book addBook(String isbn, int count, String author, String title) {
        Book book = new Book(isbn, title, author);
        for (int i = 1; i <= count; i++) {
            BookCopy c = new BookCopy(isbn + "-" + i, book);
            book.copies.add(c);
            copies.put(c.id, c);
        }
        books.put(isbn, book);
        return book;
    }

    Member addMember(String id, MembershipType type, String name) {
        Member m = new Member(id, name, type);
        members.put(id, m);
        return m;
    }

    BookCopy borrow(String memberId, String isbn, int day) {
        Member m = member(memberId);
        Book b = book(isbn);
        if (m.fine > 0) throw new LibraryException(m.id + " has an unpaid fine of Rs " + m.fine);
        if (m.atLimit())
            throw new LibraryException(m.id + " has reached the limit of " + m.type.maxBooks + " books");
        BookCopy copy = b.copyFor(m)
                .orElseThrow(() -> new LibraryException("No copy of '" + b.title + "' available"));
        copy.lendTo(m, day + m.type.loanDays);
        m.borrowed.add(copy);
        return copy;
    }

    ReturnResult returnCopy(String copyId, int day) {
        BookCopy c = copies.get(copyId);
        if (c == null || c.status != CopyStatus.BORROWED)
            throw new LibraryException(copyId + " is not borrowed");
        Member m = c.borrower;
        int late = Math.max(0, day - c.dueDay);
        int fine = late * m.type.finePerDay;
        m.fine += fine;
        m.borrowed.remove(c);
        c.returned();

        Member next = c.book.waitlist.poll();                       // required work done; now the hold
        if (next != null) {
            c.holdFor(next);
            for (LibraryListener l : listeners) l.onCopyReady(next, c);   // announce last
        }
        return new ReturnResult(late, fine);
    }

    int reserve(String memberId, String isbn) {
        Member m = member(memberId);
        Book b = book(isbn);
        if (b.waitlist.contains(m))
            throw new LibraryException(m.id + " has already reserved '" + b.title + "'");
        if (b.availableCount() > 0)
            throw new LibraryException("A copy of '" + b.title + "' is available, borrow it instead");
        b.waitlist.add(m);
        return b.waitlist.size();
    }

    int pay(String memberId, int amount) {
        Member m = member(memberId);
        int paid = Math.min(amount, m.fine);
        m.fine -= paid;
        return paid;
    }

    List<Book> search(String keyword) {
        List<Book> out = new ArrayList<>();
        for (Book b : books.values()) if (b.matches(keyword)) out.add(b);
        return out;
    }

    Member member(String id) {
        Member m = members.get(id);
        if (m == null) throw new LibraryException("No member " + id);
        return m;
    }

    Book book(String isbn) {
        Book b = books.get(isbn);
        if (b == null) throw new LibraryException("No book " + isbn);
        return b;
    }
}

public class Main {
    public static void main(String[] args) {
        LibraryService library = new LibraryService();
        NotificationOutbox outbox = new NotificationOutbox();
        library.subscribe(outbox);

        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
```

```
            try {
                switch (p[0]) {
                    case "ADD_BOOK" -> {
                        String title = String.join(" ", Arrays.copyOfRange(p, 4, p.length));
                        Book b = library.addBook(p[1], Integer.parseInt(p[2]), p[3], title);
                        StringJoiner ids = new StringJoiner(" ");
                        b.copies.forEach(c -> ids.add(c.id));
                        System.out.println("Added '" + b.title + "' by " + b.author + ", copies " + ids);
                    }
                    case "ADD_MEMBER" -> {
                        Member m = library.addMember(p[1], MembershipType.parse(p[2]), p[3]);
                        System.out.println("Member " + m.id + " " + m.name + " (" + m.type + ")");
                    }
                    case "BORROW" -> {
                        BookCopy c = library.borrow(p[1], p[2], Integer.parseInt(p[3]));
                        System.out.println(p[1] + " borrowed " + c.id + " '" + c.book.title
                                + "', due day " + c.dueDay);
                    }
                    case "RETURN" -> {
                        ReturnResult r = library.returnCopy(p[1], Integer.parseInt(p[2]));
                        if (r.daysLate() == 0) System.out.println(p[1] + " returned on time");
                        else System.out.println(p[1] + " returned " + r.daysLate()
                                + (r.daysLate() == 1 ? " day" : " days") + " late, fine Rs " + r.fine());
                    }
                    case "RESERVE" -> {
                        int position = library.reserve(p[1], p[2]);
                        System.out.println(p[1] + " reserved '" + library.book(p[2]).title
                                + "' (position " + position + ")");
                    }
                    case "PAY" -> {
                        int paid = library.pay(p[1], Integer.parseInt(p[2]));
                        System.out.println(p[1] + " paid Rs " + paid
                                + ", remaining fine Rs " + library.member(p[1]).fine);
                    }
                    case "SEARCH" -> {
                        List<Book> found = library.search(p[1]);
                        if (found.isEmpty()) System.out.println("No books match '" + p[1] + "'");
                        for (Book b : found)
                            System.out.println(b.isbn + " '" + b.title + "' by " + b.author + ", "
                                    + b.availableCount() + " of " + b.copies.size() + " available");
                    }
                    case "MEMBER" -> {
                        Member m = library.member(p[1]);
                        StringJoiner j = new StringJoiner(", ");
                        for (BookCopy c : m.borrowed) j.add(c.id + " (due day " + c.dueDay + ")");
                        String books = m.borrowed.isEmpty() ? "no books" : j.toString();
                        System.out.println(m.id + " " + m.name + " (" + m.type + "): "
                                + books + ", fine Rs " + m.fine);
                    }
                    default -> System.out.println("Unknown command: " + p[0]);
                }
            } catch (LibraryException e) {
                System.out.println(e.getMessage());
            }
            outbox.drain().forEach(System.out::println);    // notifications come after the result line
        }
    }
}
```

How to read it: each rule sits on the object that owns its data (`atLimit`, `copyFor`, `matches`). The service checks in exactly the order R3 lists. The maps in the service stand in for repositories; in a bigger design they'd be `BookRepository` and `MemberRepository` (Starter Kit, section 6).

Part C — model answers

1. STUDENT, then exam week. STUDENT is one line: `STUDENT(3, 21, 2)`. Nothing else changes. "No fines in exam week" is different: now the **formula** changes, not just a number, and it depends on the date. That's when a `FinePolicy` Strategy earns its place: a standard policy that uses the type's rate, plus an exam-week policy (or a decorator around it) that returns 0 for students during those days. The rule: **numbers differ → enum fields; behaviour differs → a strategy.**

2. Releasing a hold after 3 days. A **scheduled job**, run once a day, not an Observer, because nothing happens when a member fails to turn up. Store the day each hold started on the copy. The job finds holds older than 3 days, passes each copy to the next person on the waiting list (and notifies them), or makes it available if nobody's waiting.

3. Two desks, one last copy. Borrowing is check-then-act: "find an available copy" then "lend it". Two desks can both find the same copy and both lend it. **One server:** make that pair one step. A lock per book (so different books don't block each other), or simply a `synchronized` borrow method to start with. **Three servers:** let the database decide with a conditional update: `UPDATE copy SET status = 'BORROWED', borrower = ? WHERE id = ? AND status = 'AVAILABLE'`. One row changed means you got it; zero rows means someone else did, so try another copy or report "no copy".

4. "Why not member.borrow(book)?" "You'd be right that the member should own its own rules, and it does: `atLimit()` lives on `Member`. But borrowing touches more than the member: the book's copies, holds, the waiting list, due days and, later, notifications. If `Member.borrow` did all that, Member would need to know the inside of Book. So each object keeps its own rules, and a service coordinates them."

Common mistakes

- Counting held copies as available, in search or in borrowing by someone else.
- Checking the borrowing rules in a different order, so the wrong error appears.
- The listener printing directly, so "Notify…" appears **before** the return line.
- Strategy classes for BASIC and PREMIUM, when they differ only in numbers.
- Keeping an `available` counter in a field that drifts out of step; compute it from the copies.
- Exact text: "1 day late", but "3 days late".

Score yourself (1 point each)

☐	The entity kinds are right (classes, enums, records)
☐	Membership types are an enum with fields, and you can say why
☐	Each rule sits on the object that owns its data
☐	Borrow checks run in the required order
☐	Held copies go only to their holder, and don't count as available
☐	Fines are worked out on return and block borrowing until paid
☐	A listener interface, with the notification printed after the result
☐	Your output matches exactly
☐	Part C 1: exam week leads to a Strategy, and you said why
☐	Part C 2: hold expiry is a scheduled job, not an Observer

Solution 4 — Parking Lot

Questions a strong candidate asks first

Question	Answer assumed here
Which vehicle types, and can one use another's spot?	Car, bike, truck; each uses only its own type
How is a spot chosen? Will that change?	By a policy the operator can switch
How is the fee worked out? Any special pricing?	Hourly, rounded up, minimum 1 hour; +50% at weekends
How many gates work at the same time? One server or many?	Several gates; one server for now (Part C 3 asks about three)
What does the display show, and who else might need updates?	Free spots, FULL and AVAILABLE; more listeners later
Payment, reservations, overnight stays?	Out of scope

Part A — model answer

```
ParkingLot 1 ◆——— 1..* Floor 1 ◆——— * Spot ———> 0..1 Vehicle ———> VehicleType

ParkingService ———> ParkingLot                    Ticket ———> Vehicle,  Ticket ———> Spot
ParkingService ———> «interface» AllocationPolicy   ◁- - -  LowestFloorPolicy, MostFreeFloorPolicy
ParkingService ———> «interface» FeeCalculator      ◁- - -  HourlyFee, WeekendSurcharge (wraps a FeeCalculator)
ParkingService ———> * «interface» ParkingListener  ◁- - -  DisplayBoard
```

Need	Design	Why
Which spot, with a policy chosen by name	**Strategy + Factory**	the rule varies and is switched at run time; the name arrives as text
Hourly fee, plus a weekend layer	**Strategy + Decorator**	a surcharge wraps any base fee; a "festival discount" later is one more wrapper (NF2)
The display reacts to park and leave	**Observer**	parking doesn't know who listens (NF3)
The front door for park and unpark	**Facade**	`ParkingService` coordinates policy, lot, tickets, fees and listeners
Rates and spot letters per vehicle type	**enum with fields**, no pattern	the types differ only in numbers
The lot itself	**one instance**, created in `main` and passed in	shared, but no Singleton class needed
A spot being free or taken	a field, no State pattern	one yes/no; nothing else depends on it

NF1: what must be one indivisible step. In `park`: "is this vehicle already parked?" + "choose a free spot" + "occupy it" + "record the ticket". If any two of these can be separated, two gates can both see the same spot as free. They go inside **one** `synchronized` block on a lock object. In `unpark`: find the ticket, check the time, work out the fee, free the spot, remove the records, also as one step. Ticket numbers come from an `AtomicInteger`, taken only after a spot is secured, so failures don't use up numbers.
Outside the lock: notifying the listeners. A slow or crashing display must not hold up every gate. The board's own methods are `synchronized`, because many gates call it at once.

park	unpark
1. Read the vehicle type (throws if unknown): required 2. **Locked:** duplicate check, choose, occupy, create the ticket: required 3. Tell the listeners: announcement 4. Return the ticket	1. **Locked:** find the ticket, check the time, work out the fee, free the spot, remove the records: required 2. Tell the listeners: announcement 3. Return the exit details

Testing a race, honestly. While preparing this PDF, the solution was run with its lock deliberately removed. The RUSH test caught the bug in only **1 run out of 30**. A race test that passes proves little; the reasoning above ("these four steps are one step, under one lock") is what makes the code correct.

Part B — the code

```
import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.Supplier;
```

```
class ParkingException extends RuntimeException {
    ParkingException(String message) { super(message); }
}

// Types differ only in data (spot code, hourly rate), so an enum with fields.
enum VehicleType {
    CAR('C', 20), BIKE('B', 10), TRUCK('T', 50);

    final char spotCode;
    final int hourlyRate;
    VehicleType(char spotCode, int hourlyRate) { this.spotCode = spotCode; this.hourlyRate = hourlyRate; }

    static VehicleType parse(String s) {
        try { return valueOf(s); }
        catch (IllegalArgumentException e) { throw new ParkingException("Unknown vehicle type " + s); }
    }
}

record Vehicle(String regNo, VehicleType type) {}

class Spot {
    final String id;
    final VehicleType type;
    private Vehicle parked;                                          // null when free

    Spot(String id, VehicleType type) { this.id = id; this.type = type; }

    boolean isFree()        { return parked == null; }
    void occupy(Vehicle v)  { parked = v; }
    void release()          { parked = null; }
}

class Floor {
    final int number;
    private final Map<VehicleType, List<Spot>> spots = new EnumMap<>(VehicleType.class);

    Floor(int number, int cars, int bikes, int trucks) {
        this.number = number;
        add(VehicleType.CAR, cars);
        add(VehicleType.BIKE, bikes);
        add(VehicleType.TRUCK, trucks);
    }

    private void add(VehicleType type, int count) {
        List<Spot> list = new ArrayList<>();
        for (int i = 1; i <= count; i++) list.add(new Spot("F" + number + "-" + type.spotCode + i, type));
        spots.put(type, list);
    }

    List<Spot> spotsOf(VehicleType type) { return spots.get(type); }
    int capacity(VehicleType type)        { return spots.get(type).size(); }
    int freeCount(VehicleType type) {
        return (int) spots.get(type).stream().filter(Spot::isFree).count();
    }

    Optional<Spot> firstFree(VehicleType type) {
        return spots.get(type).stream().filter(Spot::isFree).findFirst();
    }
}

// The building itself: created once in main and passed in where it's needed.
class ParkingLot {
    private final List<Floor> floors = new ArrayList<>();
    ParkingLot(int floorCount, int cars, int bikes, int trucks) {
        for (int f = 1; f <= floorCount; f++) floors.add(new Floor(f, cars, bikes, trucks));
    }
    List<Floor> floors() { return floors; }
}

// ---------- Strategy: which spot to give ----------
interface AllocationPolicy {
    Optional<Spot> choose(List<Floor> floors, VehicleType type);
}

class LowestFloorPolicy implements AllocationPolicy {
    public Optional<Spot> choose(List<Floor> floors, VehicleType type) {
        for (Floor f : floors) {
            Optional<Spot> s = f.firstFree(type);
            if (s.isPresent()) return s;
        }
        return Optional.empty();
    }
}

class MostFreeFloorPolicy implements AllocationPolicy {
    public Optional<Spot> choose(List<Floor> floors, VehicleType type) {
        Floor best = null;
        for (Floor f : floors) {             // strictly more wins, so a tie keeps the lower floor
            int free = f.freeCount(type);
            if (free > 0 && (best == null || free > best.freeCount(type))) best = f;
        }
        return best == null ? Optional.empty() : best.firstFree(type);
    }
}

// ---------- Factory: the policy arrives as text ----------
class PolicyFactory {
    private static final Map<String, Supplier<AllocationPolicy>> POLICIES = Map.of(
            "LOWEST_FLOOR", LowestFloorPolicy::new,
            "MOST_FREE_FLOOR", MostFreeFloorPolicy::new);
```

```
    static AllocationPolicy create(String name) {
        Supplier<AllocationPolicy> s = POLICIES.get(name);
        if (s == null) throw new ParkingException("Unknown policy " + name);
        return s.get();
    }
}

// ---------- Strategy + Decorator: what to charge ----------
interface FeeCalculator {
    int fee(VehicleType type, int minutes);
}

class HourlyFee implements FeeCalculator {
    static int billableHours(int minutes) { return Math.max(1, (minutes + 59) / 60); }
    public int fee(VehicleType type, int minutes) { return billableHours(minutes) * type.hourlyRate; }
}

class WeekendSurcharge implements FeeCalculator {
    private final FeeCalculator inner;
    WeekendSurcharge(FeeCalculator inner) { this.inner = inner; }
    public int fee(VehicleType type, int minutes) { return inner.fee(type, minutes) * 150 / 100; }
}

record Ticket(String id, Vehicle vehicle, Spot spot, int entryMinute) {}

record Exit(Ticket ticket, int hours, int fee) {}

// ---------- Observer: who reacts to park / leave ----------
interface ParkingListener {
    void onParked(Ticket ticket);
    void onLeft(Ticket ticket);
}

class DisplayBoard implements ParkingListener {
    private final Map<Integer, Map<VehicleType, Integer>> freeByFloor = new TreeMap<>();
    private final Map<VehicleType, Integer> freeTotal = new EnumMap<>(VehicleType.class);
    private final List<String> announcements = new ArrayList<>();   // printed after the command's result

    DisplayBoard(ParkingLot lot) {
        for (Floor f : lot.floors()) {
            Map<VehicleType, Integer> m = new EnumMap<>(VehicleType.class);
            for (VehicleType t : VehicleType.values()) {
                m.put(t, f.capacity(t));
                freeTotal.merge(t, f.capacity(t), Integer::sum);
            }
            freeByFloor.put(f.number, m);
        }
    }

    private static int floorOf(Spot s) { return Integer.parseInt(s.id.substring(1, s.id.indexOf('-'))); }

    public synchronized void onParked(Ticket t) {
        VehicleType type = t.vehicle().type();
        freeByFloor.get(floorOf(t.spot())).merge(type, -1, Integer::sum);
        int left = freeTotal.merge(type, -1, Integer::sum);
        if (left == 0) announcements.add("Display: " + type + " FULL");
    }

    public synchronized void onLeft(Ticket t) {
        VehicleType type = t.vehicle().type();
        freeByFloor.get(floorOf(t.spot())).merge(type, 1, Integer::sum);
        int left = freeTotal.merge(type, 1, Integer::sum);
        if (left == 1) announcements.add("Display: " + type + " AVAILABLE");
    }

    synchronized List<String> drain() {
        List<String> out = new ArrayList<>(announcements);
        announcements.clear();
        return out;
    }

    synchronized String freeReport() {
        StringBuilder sb = new StringBuilder();
        freeByFloor.forEach((floor, m) -> sb.append("F").append(floor)
                .append(": car ").append(m.get(VehicleType.CAR))
                .append(", bike ").append(m.get(VehicleType.BIKE))
                .append(", truck ").append(m.get(VehicleType.TRUCK)).append('\n'));
        return sb.toString();
    }
}

// ---------- Facade: the one front door for park and unpark ----------
class ParkingService {
    private final ParkingLot lot;
    private final Object lock = new Object();
    private final Map<String, Ticket> activeTickets = new HashMap<>();      // guarded by lock
    private final Set<String> parkedRegs = new HashSet<>();                  // guarded by lock
    private final AtomicInteger nextTicket = new AtomicInteger(1);
    private final List<ParkingListener> listeners = new CopyOnWriteArrayList<>();
    private volatile AllocationPolicy policy = new LowestFloorPolicy();
    private volatile FeeCalculator fees = new HourlyFee();

    ParkingService(ParkingLot lot) { this.lot = lot; }

    void subscribe(ParkingListener l)       { listeners.add(l); }
    void setPolicy(AllocationPolicy p)      { policy = p; }
    void setWeekend(boolean weekend) {
        fees = weekend ? new WeekendSurcharge(new HourlyFee()) : new HourlyFee();
    }
```

```
    Ticket park(String typeName, String regNo, int minute) {
        VehicleType type = VehicleType.parse(typeName);
        Vehicle vehicle = new Vehicle(regNo, type);
        Ticket ticket;
        synchronized (lock) {                                   // check-then-act as ONE step
            if (parkedRegs.contains(regNo)) throw new ParkingException(regNo + " is already parked");
            Spot spot = policy.choose(lot.floors(), type)
                    .orElseThrow(() -> new ParkingException("No " + type + " spot available for " + regNo));
            spot.occupy(vehicle);
            parkedRegs.add(regNo);
            ticket = new Ticket("T" + nextTicket.getAndIncrement(), vehicle, spot, minute);
            activeTickets.put(ticket.id(), ticket);
        }
        for (ParkingListener l : listeners) l.onParked(ticket);   // announce last, outside the lock
        return ticket;
    }

    Exit unpark(String ticketId, int minute) {
        Ticket t;
        int fee;
        synchronized (lock) {
            t = activeTickets.get(ticketId);
            if (t == null) throw new ParkingException("Invalid ticket " + ticketId);
            if (minute < t.entryMinute())
                throw new ParkingException("Exit time " + Clock.format(minute)
                        + " is before entry " + Clock.format(t.entryMinute()));
            fee = fees.fee(t.vehicle().type(), minute - t.entryMinute());
            activeTickets.remove(ticketId);
            parkedRegs.remove(t.vehicle().regNo());
            t.spot().release();
        }
        for (ParkingListener l : listeners) l.onLeft(t);
        return new Exit(t, HourlyFee.billableHours(minute - t.entryMinute()), fee);
    }
}

class Clock {
    static int parse(String hhmm) {
        String[] p = hhmm.split(":");
        return Integer.parseInt(p[0]) * 60 + Integer.parseInt(p[1]);
    }
    static String format(int minute) { return String.format("%02d:%02d", minute / 60, minute % 60); }
}

public class Main {
    public static void main(String[] args) throws Exception {
        ParkingService service = null;
        DisplayBoard board = null;
        Scanner in = new Scanner(System.in);
        while (in.hasNextLine()) {
            String line = in.nextLine().trim();
            if (line.isEmpty()) continue;
            String[] p = line.split("\\s+");
            try {
                if (!p[0].equals("CREATE_LOT") && service == null)
                    throw new ParkingException("No parking lot created");
                switch (p[0]) {
                    case "CREATE_LOT" -> {
                        int floors = Integer.parseInt(p[1]), cars = Integer.parseInt(p[2]);
                        int bikes = Integer.parseInt(p[3]), trucks = Integer.parseInt(p[4]);
                        ParkingLot lot = new ParkingLot(floors, cars, bikes, trucks);
                        service = new ParkingService(lot);
                        board = new DisplayBoard(lot);
                        service.subscribe(board);
                        System.out.println("Created lot: " + floors + " floors, each with "
                                + cars + " car, " + bikes + " bike, " + trucks + " truck spots");
                    }
                    case "PARK" -> {
                        Ticket t = service.park(p[1], p[2], Clock.parse(p[3]));
                        System.out.println("Ticket " + t.id() + ": " + t.vehicle().type() + " "
                                + t.vehicle().regNo() + " at " + t.spot().id
                                + ", in " + Clock.format(t.entryMinute()));
                    }
                    case "UNPARK" -> {
                        Exit e = service.unpark(p[1], Clock.parse(p[2]));
                        Ticket t = e.ticket();
                        System.out.println(t.id() + ": " + t.vehicle().type() + " " + t.vehicle().regNo()
                                + " left " + t.spot().id + " after " + e.hours() + " h, fee Rs " + e.fee());
                    }
                    case "POLICY" -> {
                        service.setPolicy(PolicyFactory.create(p[1]));
                        System.out.println("Policy: " + p[1]);
                    }
                    case "DAY" -> {
                        if (!p[1].equals("WEEKDAY") && !p[1].equals("WEEKEND"))
                            throw new ParkingException("Unknown day " + p[1]);
                        service.setWeekend(p[1].equals("WEEKEND"));
                        System.out.println("Day: " + p[1]);
                    }
                    case "FREE" -> System.out.print(board.freeReport());
                    case "RUSH" -> {
                        int count = Integer.parseInt(p[1]);
                        String type = p[2];
                        int minute = Clock.parse(p[3]);
                        ParkingService s = service;
                        AtomicInteger parked = new AtomicInteger(), turnedAway = new AtomicInteger();
                        CountDownLatch startGun = new CountDownLatch(1);
                        ExecutorService pool = Executors.newFixedThreadPool(count);
                        for (int i = 1; i <= count; i++) {
                            String reg = "RUSH-" + i;
                            pool.submit(() -> {
```

```
                            try {
                                startGun.await();                       // everyone starts together
                                s.park(type, reg, minute);
                                parked.incrementAndGet();
                            } catch (ParkingException e) {
                                turnedAway.incrementAndGet();
                            } catch (InterruptedException e) {
                                Thread.currentThread().interrupt();
                            }
                        });
                    }
                    startGun.countDown();
                    pool.shutdown();
                    pool.awaitTermination(10, TimeUnit.SECONDS);
                    System.out.println("Rush: " + parked.get() + " parked, "
                            + turnedAway.get() + " turned away");
                }
                default -> System.out.println("Unknown command: " + p[0]);
            }
        } catch (ParkingException e) {
            System.out.println(e.getMessage());
        }
        if (board != null) board.drain().forEach(System.out::println);   // display lines come last
    }
}
```

How to read it: `ParkingService.park()` follows the step list above exactly. `MostFreeFloorPolicy` keeps the lower floor on a tie because it only replaces its choice when a floor has *strictly* more free spots. The display board collects its announcements in a list; the driver prints them after each command's result, which also keeps the RUSH output in a fixed order.

Part C — model answers

1. Bikes in car spots. Give each vehicle type an ordered list of spot types it may use: `BIKE → [BIKE, CAR]`, `CAR → [CAR]`. The policies try each allowed spot type in order. The fee stays on the **vehicle** type, so a bike in a car spot still pays Rs 10. One subtlety to raise: the display counts are by **spot** type, so a bike in a car spot reduces the *car* count. Spot type and vehicle type are now different things, and the code must stop treating them as one.

2. EV charging. A new spot type, EV, plus a `ChargingFee` decorator that adds Rs 15 × hours when the ticket's spot is an EV spot. Notice the decorator needs to know the spot, not just the vehicle type and minutes. That's a sign `FeeCalculator.fee(...)` should take the `Ticket`, which is worth saying in the interview: the design improves when a real requirement arrives.

3. Three servers. `synchronized` protects one JVM. With three servers, two of them can hand out the same spot. Move the decision to the shared store: a conditional update, `UPDATE spot SET reg = ? WHERE id = ? AND reg IS NULL`, where one row changed means you got the spot; a unique constraint so one number plate can't have two active tickets; ticket numbers from a database sequence. Redis with `SETNX` on the spot id is the other common answer.

4. Payment fails at the barrier. Paying is a required step. The spot is **not** freed, the ticket stays active, and the barrier stays down. The driver can retry or pay another way, and the display doesn't announce anything, because nothing changed. Only after payment succeeds: free the spot, close the ticket, then announce.

5. "Why an Observer for one display?" "You'd be right if the display were the only thing that will ever care. But parking shouldn't know displays exist: the mall already wants an entrance board *and* a mobile app showing free spots, and analytics will follow. With listeners, each of those is a new class and `park()` doesn't change. A display that crashes also can't break parking, because it's called after the spot is secured and outside the lock."

Common mistakes

- The duplicate check, spot choice and occupy in **separate** locked sections, or only some of them locked.
- Calling listeners while holding the lock.
- Taking a ticket number before the spot is secured, so failed attempts leave gaps.
- A display that keeps its own counts but misses unpark events, so FREE drifts.
- Fees: forgetting the 1-hour minimum; rounding with division before adding 59; applying the weekend rate at entry instead of exit.
- A fee Strategy class per vehicle type, when they differ only in their rate.
- A Singleton class (Bill Pugh) for the lot, instead of creating it once and passing it in.

Score yourself (1 point each)

☐	Entities and multiplicities are right on your diagram
☐	The allocation policy is a Strategy, created from its name by a factory
☐	The weekend surcharge wraps the base fee (Decorator)
☐	The display is a listener; park and unpark don't know about it
☐	Vehicle rates are enum fields, not strategy classes
☐	The duplicate check, choose, occupy and ticket happen under one lock
☐	Listeners are notified after the lock, and announcements print after the result
☐	Ticket numbers stay in sequence, with no gaps after failures
☐	Your output matches exactly, RUSH included
☐	Part C 3: you named a database or Redis answer for several servers

Back pages

Warm-up answers

#	Answer	Why
1	**Strategy**	A different formula per plan, changing yearly. (If the plans differed only in their prices, an enum with fields would do.)
2	**Observer**	Reactions to a cancellation that must not undo it. One nuance worth saying aloud: the **refund** is arguably a required step, not a side effect. If it fails, a member is out of pocket, so call it directly, or make sure it's retried.
3	**State**	Five operations, each depending on the status.
4	**Nothing**: a scheduled job	Nothing happens when a member stays away, so there's no event. Run a daily job.
5	**Adapter**	A vendor's shape that we can't edit.
6	**Decorator**	Extras that stack in any combination.
7	**Chain of Responsibility**	Up to X this one decides, up to Y the next.
8	**Facade**	Several steps in order, and three callers.
9	**Builder**	Required plus many optional fields, never changing after creation.
10	**Nothing**: a record	Just two fields.
11	**Prototype**	Copy a finished plan, then change a little.
12	**Mediator**	The screen coordinates its boxes; the boxes don't talk to each other.
13	**Visitor**	The kinds are fixed; new operations keep coming.
14	**Null Object**	Missing is normal, and "show nothing" is the right behaviour.

12 or more right: recognition is solid. Under 10: re-read the cheat sheet in the Starter Kit before PDF 2.

Mistake log

One row per mistake, written in your own words. Before starting PDF 2, read this page once: these are the mistakes most likely to come back.

Date	Problem	What I did	What's right	Rule to remember

Re-do schedule

Problem	Attempted	Day 3: Part A from memory (15 min)	Day 10: Part A + the core class	Day 30: the whole problem, timed	Score /10
Tic-Tac-Toe					
Vending Machine					
Library Management					
Parking Lot					

Next: PDF 2. Deck of Cards, ATM, Splitwise and the Elevator System. Its warm-up will ask you to re-design two problems from this PDF from memory, so the day-10 re-dos above are worth doing before you open it.